

REFLECT ON PROGRAMMING LANGUAGE

Matin Mohammadi, Robert Hanssen, Sander Gnanavel

IKT467-1 25H

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja / Nei
2.	Vi erklærer videre at denne besvarelsen: • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja / Nei
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høyskoler i Norge, jf. Universitets- og høyskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja / Nei
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja / Nei
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høyskolens retningslinjer for behandling av saker om fusk.	Ja / Nei
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja / Nei
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Ja / Nei

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).

Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Ja / Nei
Er oppgaven båndlagt (konfidensiell)?	Ja / Nei
Er oppgaven unntatt offentlighet?	Ja / Nei

Contents

List of Figures	ii
List of Tables	ii
0.1 Tools	1
0.2 Programming language. Which implementation language would fit best? What are the alternatives? What experiments did you run? How did you decide?	2
0.3 User interface. How do you envision the user interface (sketches / wireframes)? How does your language support this?	3
0.4 Data collection. Which data do you need to collect? How is the data stored? How does your language support collection and storage?	4
0.4.1 Purpose and scope	4
0.4.2 Objects	4
0.4.3 Data to be Collected	4
0.4.4 How the Data is Stored	5
0.4.5 Python support	5
0.5 Data protection. How should the data be protected? What mechanisms does your language provide for data protection?	6
0.6 Data analysis. How do you need to analyse the collected data? How does your language help?	7
0.6.1 How we compare with SUMO (our referent system)	7
0.6.2 Programming language	7
0.7 Runtime. Which runtime structures are implied by your language? Which of those are visible from your perspective?	8
Bibliography	9

0.1 Tools

- ChatGPT: used for grammar editing and improving writing style
- Overleaf: used for writing and formatting the report

0.2 Programming language. Which implementation language would fit best? What are the alternatives? What experiments did you run? How did you decide?

When we started thinking about which programming language to use for our simulation, we were curious about whether GoDot might be a good option. Its strong visualization features and reputation for building interactive and graphical environments made it an appealing choice, especially since our project is not just about calculating vehicle flows and driver behavior but also about showing clear visualizations like time–space diagrams and scenario comparisons. GoDot seemed like it could help us create engaging and dynamic results.

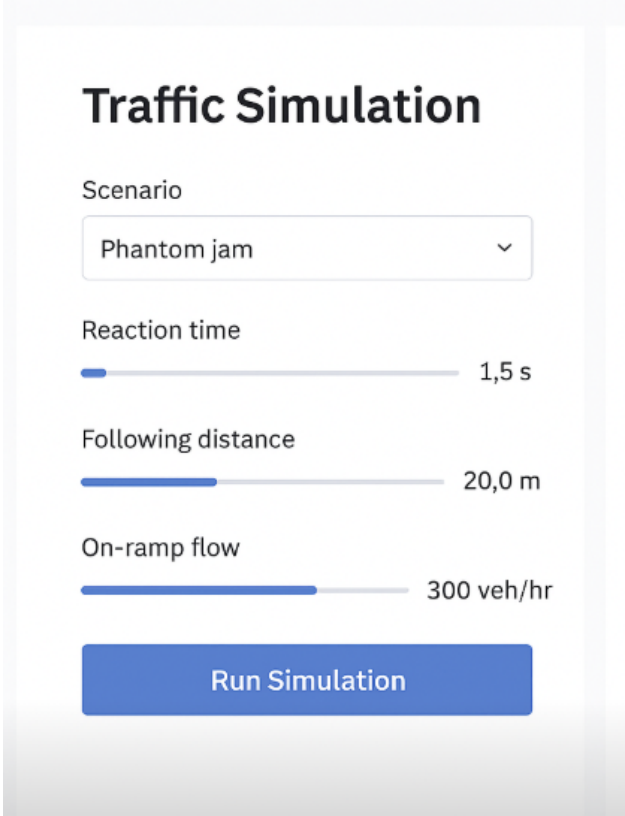
We also considered C++, since it is well known for its speed and efficiency. For large-scale simulations with many vehicles and high computational demands, C++ could have given us strong performance and low-level control over how the simulation is executed. However, we realized that the extra complexity of developing and maintaining the project in C++, combined with the difficulty of quickly building analysis and visualization tools, would not serve our current goals as well.

As we looked more closely at what the project really needs, Python emerged as the best fit. It comes with a rich ecosystem of scientific libraries for computation, data analysis, and plotting, and it integrates well with SUMO and TraCI, which we rely on for validation against real traffic data. Python makes it easier to build the simulation core, work with time-series outputs at the resolution we require, and generate the plots and aggregated metrics that form the backbone of our analysis. While GoDot might offer more advanced visualization options, and C++ might provide higher performance, the added effort of connecting them with our analytical tools and external simulators would have made the development process unnecessarily complicated. In the end, we chose Python because it strikes the right balance between development speed, analytical power, and support for the validation process that is so central to our project.

0.3 User interface. How do you envision the user interface (sketches / wireframes)? How does your language support this?

For the user interface, we picture something simple, clear, and easy to use. The idea is that anyone should be able to set up a simulation, run it, and then explore the results without needing to dig into the code. At the start, the interface would let the user choose between the main scenarios we are studying, such as the phantom jam, driver variation, or bottleneck. Once a scenario is selected, the user could adjust important parameters like reaction time, following distance, or on-ramp flow through sliders or input fields. After running the simulation, the results would be shown directly in the same interface, with time-space diagrams of vehicle movement alongside key performance measures like flow, density, and average speed.

To keep things lightweight and connected to the simulation itself, we plan to build the first version of this interface using Streamlit in Python. Streamlit makes it easy to turn simulation inputs into sliders, checkboxes, or dropdown menus, and it can directly display plots generated with libraries such as matplotlib or Plotly. This way, we can quickly move from running a scenario to seeing the results in a visual and interactive format. If we later want to create a more polished or public-facing version, it would also be possible to connect the Python backend to a modern web framework, but for now the focus is on making sure the interface directly supports experimentation, visualization, and comparison of results. Python's ecosystem makes this straightforward, which is one of the reasons it fits our project so well.



Traffic Simulation

Scenario

Phantom jam

Reaction time

1,5 s

Following distance

20,0 m

On-ramp flow

300 veh/hr

Run Simulation

Figure 1: Prototype user interface for the traffic simulation

0.4 Data collection. Which data do you need to collect? How is the data stored? How does your language support collection and storage?

0.4.1 Purpose and scope

Traffic congestion is a complex, emergent phenomenon which impacts productivity, environmental quality, and public well-being. While often attributed to static bottlenecks or accidents, many forms of congestion, such as "phantom jams" and "stop-and-go waves" are not caused by a fixed physical obstruction but emerge from individual driver interactions and the inherent limitations of human perception, reaction time, and the resulting over-corrections in dense traffic.

To analyze such phenomenon we collect data to compute traffic-flow metrics, compare scenarios with different rates of autonomous vehicles (0%, 25%, 50%, 75%, 100%), and validate model correctness by comparing our executions against a referent system *SUMO* [2]. The collection plan includes metrics for established traffic flow relationships (fundamental diagram $q-k-v$ [6]).

0.4.2 Objects

For reproducibility each run should use random seed and a deterministic `run_id` derived from its configuration.

Objects:

- **Vehicle:** `vehicle_id`, `vehicle_length_m`, `driver_type`, `max_accel_mps2`, `max_decel_mps2`.
- **Human driver:** `aggression`, `gap_s`, `reaction_time_s`.
- **Autonomous driving:** `communication_delay`, `gap_s`, `reaction_time_s`.
- **Road:** `road_id`, `segment_length_m`, `speedlimit_mps`.

0.4.3 Data to be Collected

All times are in seconds, lengths in meters, speeds in m/s (converted to km/h only for plotting).

Input Data:

column	type	unit	description
<code>v_length_m</code>	float	m	define length of cars
<code>v_params</code>	object	–	attributes for vehicles
<code>h_params</code>	object	–	parameters for human driver behaviour
<code>a_params</code>	object	–	parameters for autonomous vehicles
<code>gap_s</code>	float	s	average gap between drivers
<code>driver_type</code>	categorical	–	human/autonomous
<code>highway_len_m</code>	categorical	m	length of highway segment
<code>speed_lim_mps</code>	float	m/s	speed limit of highway segment
<code>v_density</code>	int	vehicles/km	vehicles per kilometer of highway
<code>p_autonomous</code>	float	–	ratio of autonomous vehicles
<code>sim_dur_s</code>	float	s	duration of simulation

Output Data:

To compare the performance of different simulation scenarios, a set of KPIs will be calculated from the output data. These metrics provide an aggregate view of system efficiency, mobility, and reliability.

metric	type	unit	description
Throughput	float	vehicles/h	vehicles that exit the highway per hour
Travel Time (avg)	float	s	measures average mobility
Travel Delay (total)	float	–	actual travel time vs. free-flow travel time
Traffic Density (avg)	float	–	average traffic density

0.4.4 How the Data is Stored

To ensure reproducibility, accessibility, and efficient analysis, all data associated with a simulation run will be stored in a dedicated directory using standard, open-source file formats. The web UI will be responsible for reading these files to populate visualizations and display results.

- **Vehicle Data:** Vehicle's state (position, speed, acceleration) at each timestep will be stored in dictionaries or using Parquet format.
- **Road State Data:** Will be stored in csv format. vehicle density, throughput, average speed etc. over time.

0.4.5 Python support

Python provides robust support for this storage architecture. The PyYAML library will be used to parse and write configuration files. The Pandas library, integrated with the pyarrow engine, offers high-performance functions to read and write both CSV and Parquet files, making it straightforward to manage data I/O between the simulation core, the stored files, and the analysis scripts.

0.5 Data protection. How should the data be protected? What mechanisms does your language provide for data protection?

The project handles three main types of data: scenario configurations (such as reaction times, following distances, and random seeds), simulation traces (including vehicle ID, time, position, speed, and acceleration), and derived aggregates (such as flow, density, and average speed). While these datasets are not directly personal, detailed trajectories could become identifying if linked with external information [3]. For that reason, all raw traces are treated as sensitive.

The main goals for data protection are to maintain confidentiality of raw traces and secrets, to ensure the integrity of code, configurations, and outputs, and to guarantee availability and reproducibility of experiments [7].

Python and its ecosystem provide several mechanisms to achieve these goals. Files can be stored in project-specific directories with limited operating system permissions to prevent unintended access. Sensitive information, such as tokens or connection strings, can be loaded through environment variables rather than being hard-coded in the source code. When data must be shared or stored externally, Python libraries such as cryptography allow encryption of files so that only authorized parties with the correct keys can access them [1]. When retrieving external datasets or exposing the user interface beyond localhost, communication should always happen over HTTPS, ensuring encryption in transit.

Data minimization is applied by using synthetic identifiers for vehicles and publishing only aggregated results such as flow and density, rather than raw vehicle trajectories, unless absolutely necessary. To ensure reproducibility without risking leakage, configurations and random seeds are versioned, while raw data, outputs, and environment files are excluded from version control. Integrity can be checked by calculating and logging hashes of important configurations and result files, which can later be verified to detect tampering or errors. In cases where SUMO is used together with Python through the TraCI protocol, the control port is restricted to localhost or secured through a VPN to avoid exposure on public networks [4]. Finally, Python’s logging facilities are used to record metadata such as configuration hashes, software versions, seeds, start and end times, and output locations for each run. This ensures that every experiment is traceable and auditable.

To operationalize these measures, we maintain a simple checklist: raw data and outputs are excluded from version control, secrets are stored only in environment variables, shared artifacts are aggregated or anonymized, and every run records its configuration, seed, and version information alongside the produced outputs. Periodic offline backups of configurations and derived results are also maintained, while raw traces are encrypted if stored in cloud services.

0.6 Data analysis. How do you need to analyse the collected data? How does your language help?

The data is analyzed to quantify the performance of a traffic system under different scenarios and ratios of autonomous vehicles. Secondly, to explain why any observed differences in performance occur, and third, to formally establish the model's correctness by comparing its outputs to a trusted referent system.

In the context of this model, congestion is not a binary state but a measurable degradation of system performance. A state of congestion will be identified by tracking key metrics from traffic engineering literature. When Travel Time Index (TTI) rises and Traffic Density exceeds a critical threshold, while Throughput (Flow Rate) is decreasing, we label the period as *congested*.

1. **Travel Time Index (TTI):** This is a primary mobility measure defined as the ratio of the average peak travel time to an ideal off-peak (free-flow) travel time.
2. **Throughput (Flow Rate):** Measured in vehicles per hour (vph), this metric quantifies the number of vehicles passing the end of the highway segment. A sustained drop in throughput below the theoretical roadway capacity indicates a breakdown into an oversaturated.
3. **Traffic Density:** Measured in vehicles per kilometer per lane (veh/km/lane), this metric indicates the proximity of vehicles. When density exceeds a threshold the traffic stream becomes unstable and susceptible to congestion.

0.6.1 How we compare with SUMO (our referent system)

We compare results from our data analysis with SUMO's, and if the shapes and peaks match within a reasonable range (roughly $\pm 10\%$ for flow and a few km/h for speed), we count that as agreement and investigate any larger gaps.

0.6.2 Programming language

The Python programming language has many useful supporting libraries and the primary analytical visualizations will be generated using the *Matplotlib* and *Seaborn* libraries, which offer extensive plotting capabilities. The calculation of KPIs will be handled using the numerical processing libraries *NumPy* and *Pandas*.

0.7 Runtime. Which runtime structures are implied by your language? Which of those are visible from your perspective?

Using Python as the main implementation language brings with it a number of underlying runtime structures. The simulation itself runs inside the CPython interpreter, which is responsible for executing bytecode, handling the global interpreter lock, memory allocation, and garbage collection [5]. The traffic simulation is organised as a discrete-time loop with fixed timesteps, for example one update every 100 milliseconds. At each tick the state of every vehicle, such as position, speed, and acceleration, is updated and recorded as part of the trace.

When SUMO is used for validation, Python communicates with a separate SUMO process through the TraCI protocol over a TCP socket. This creates a multi-process runtime where Python acts as the client and SUMO functions as the server [4]. During each run, data is held in memory using structures such as NumPy arrays and pandas DataFrames, which are optimised in C but exposed through Python. If a user interface is included, for example through Streamlit, an additional long-running server process is started which re-executes the Python script whenever the user interacts with the interface and caches results to speed up performance.

From the perspective of the developer and user, several of these structures are directly observable. The simulation loop itself is visible in execution through progress bars, log output, and the generated results. External processes such as SUMO appear in the operating system's process list and their connection status is visible through TraCI. A Streamlit application runs as a local web server, which can be seen in the browser and in the terminal log. The files and plots produced during each run, such as CSV tables and time-space diagrams, also make the runtime execution tangible.

Other parts of the runtime remain hidden. The global interpreter lock, garbage collector, memory manager, and the internal C-loops that power numerical libraries are not directly visible. Their effects can only be observed indirectly through execution speed, responsiveness of the interface, or the amount of memory consumed.

In summary, while the low-level mechanisms of the Python interpreter are abstracted away, the developer mainly interacts with the visible loop, the orchestration of processes between Python and SUMO, and the outputs that are produced. The runtime perspective therefore consists of a mixture of hidden interpreter internals and observable structures such as loops, processes, servers, and files.

Bibliography

- [1] The Cryptography Developers. *Cryptography Library for Python*. 2023. URL: <https://cryptography.io/>.
- [2] *Eclipse SUMO: Simulation of Urban MObility*. <https://eclipse.dev/sumo/>. Version 1.24.0 (accessed YYYY-MM-DD). Eclipse Foundation, 2025.
- [3] *General Data Protection Regulation (GDPR)*. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [4] Pablo A. Lopez et al. “SUMO – Simulation of Urban MObility: An Overview.” In: *Proceedings of the 21st International Conference on Intelligent Transportation Systems*. 2018.
- [5] Python Software Foundation. *Python Documentation: CPython Runtime*. 2023. URL: <https://docs.python.org/3/c-api/intro.html>.
- [6] Aleksandra Romanowska and Kazimierz Jamroz. “Comparison of Traffic Flow Models with Real Traffic Data Based on a Quantitative Assessment.” In: *Applied Sciences* 11.21 (2021). ISSN: 2076-3417. DOI: [10.3390/app11219914](https://doi.org/10.3390/app11219914). URL: <https://www.mdpi.com/2076-3417/11/21/9914>.
- [7] William Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, 2017.